VCLUSTER APPLICATION

# Technical Documentation Writing Samples

**Declan T. Goodwin | Technical Writer and Information Architect**

This packet combines a public software-learning project with two sanitized professional examples. Together, they demonstrate conceptual and procedural writing, information architecture, hands-on system investigation, audience-specific guidance, and documentation that helps users complete complex work.

## What this packet demonstrates

| CAPABILITY | EVIDENCE IN THIS PACKET |
|---|---|
| Conceptual documentation | Layered explanations, product boundaries, terminology, and decision guidance for readers with different levels of technical experience. |
| Procedural documentation | Step-based guidance, prerequisites, ownership, dependencies, branching paths, and completion evidence. |
| Information architecture | Repeatable content models, progressive disclosure, scannable tables, and role-specific entry points. |
| Hands-on verification | Systems investigated, configured, tested, troubleshot, and documented from direct use rather than secondhand notes. |
| Version-controlled publishing | Public documentation built and published through GitHub Pages, with iterative revisions and visible source history. |

## Included samples

01

### LangChain Learning Site

Public conceptual documentation built for beginner, intermediate, and progressively technical audiences.

02

### Role-Based SOP Suite

Six onboarding and offboarding guides with branching pathways, explicit ownership, and more than 230 actions.

03

### Automated Intake Workflow

Embedded guidance, confirmations, status messages, exception handling, and administrator recovery steps.

*Sanitization note: The professional examples were created in a federal contracting environment. Organization-specific names, addresses, repository paths, system identifiers, access details, and controlled information have been removed or replaced with representative language. The structures and writing decisions reflect the original deliverables.*

SAMPLE 01

# LangChain Learning Documentation Site

| | |
|---|---|
| **PROBLEM** | AI product documentation can assume too much prior knowledge or add terminology faster than it builds understanding. |
| **MY ROLE** | Researched the product ecosystem, designed the content architecture, wrote and revised explanations, created diagrams, and published the site. |
| **AUDIENCE** | Readers moving from general AI familiarity toward practical understanding of agent frameworks and orchestration. |
| **TOOLS** | GitHub, GitHub Pages, HTML, CSS, Markdown, diagrams, and AI-assisted research and revision. Beginner and intermediate explanations are published; advanced content is in development. |

## Public project

declan-goodwin.github.io/-Dgoodwin-Learning-Chain-Gang/whats-langchain

## Content architecture

| BEGINNER | INTERMEDIATE | ADVANCED |
|---|---|---|
| **Build the mental model** | **Explain the system** | **Expose design choices** |
| What LangChain is, what problem it solves, who uses it, and when a simpler approach may be better. | Models, messages, tools, agents, state, memory, middleware, structured output, and product relationships. | Execution control, persistence, failure handling, evaluation, deployment, and implementation tradeoffs. In development. |

## Documentation approach

- Define unfamiliar terms before relying on them, then reuse terminology consistently as technical depth increases.
- Separate LangChain, LangGraph, and LangSmith responsibilities so readers understand abstraction, orchestration, and observability.
- Use progressive disclosure, expandable sections, diagrams, and repeated patterns to keep dense material navigable.
- State when not to use the product, including cases where a direct model call or deterministic workflow is sufficient.

## Representative product boundaries

- LangChain: higher-level components and agent abstractions for assembling tool-using applications.
- LangGraph: lower-level orchestration, state, persistence, and controlled execution.
- LangSmith: tracing, evaluation, and observability for inspecting and improving behavior.

SAMPLE 01 | ADAPTED EXCERPT

# Choosing the Right Level of Control

LangChain is useful when an application needs a model to work with tools, data, and reusable components. It is not automatically the right answer for every AI feature. The amount of flexibility the application needs should determine the amount of framework and orchestration you introduce.

## Decision guide

| NEED | APPROACH | WHY |
|---|---|---|
| One prompt and one response | Direct model call | The task has no tool use, persistent state, or multi-step coordination. |
| A known sequence of steps | Deterministic workflow | The order is fixed and predictable behavior matters more than model-selected actions. |
| Flexible tool selection | LangChain agent | The model must choose tools or next steps based on the request and intermediate results. |
| Explicit state and recovery | LangGraph workflow | The application needs controlled transitions, checkpoints, persistence, or human review. |
| Tracing and evaluation | LangSmith | The team needs to inspect runs, compare behavior, and measure changes over time. |

## State, context, and memory

These terms are related, but they are not interchangeable. Keeping the distinctions clear helps readers reason about what the application knows, what the model can see, and what information survives beyond the current step.

| TERM | PRACTICAL MEANING | KEY QUESTION |
|---|---|---|
| State | The working record that workflow nodes read and update during a run. | What has happened, and what should happen next? |
| Context | The information exposed to the model for the current call. | What information can the model use right now? |
| Memory | Information retained for later interactions, often through a checkpointer or store. | What should remain available after this step or session? |

Practical loop: frame the problem, choose the simplest workable architecture, define tools and instructions, test and evaluate behavior, then monitor and revise. The excerpt connects terminology to decisions and states limits instead of treating the framework as a universal solution.

**View the live project:** What Is LangChain?

SAMPLE 02

# Role-Based Onboarding and Offboarding SOP Suite

| | |
|---|---|
| **PROBLEM** | Civilian, contractor, and military personnel followed overlapping but nonidentical paths, with responsibilities distributed across multiple offices. |
| **MY ROLE** | Mapped the process, separated user pathways, assigned step ownership, documented dependencies and concurrent actions, and authored the full suite. |

## Deliverable

Six role-specific check-in and check-out SOPs comprising more than 230 documented actions. The suite was selected for division-wide adoption.

## Why the documentation was split into six guides

A universal SOP would have forced readers to interpret irrelevant conditions. Dividing the material by personnel category and process direction gave each reader a shorter accurate path while preserving a shared structure. Civilian and military guides included additional entry-path branches.

## Representative procedure

| STEP | OWNER | ACTION | DEPENDENCY OR EVIDENCE |
|---|---|---|---|
| 1.1 Initiate | Operations lead | Provide the effective date, personnel category, work location, and known access requirements. | Approved action or confirmed date |
| 1.2 Route | Process coordinator | Verify existing enterprise identities and profiles. Select the abbreviated or full pathway. | Identity and account status |
| 1.3 Issue | Process coordinator | Send the role-specific checklist, forms, training instructions, due dates, and submission method. | Correct checklist selected |
| 1.4 Coordinate | Process owners | Begin eligible access, equipment, badging, repository, and security actions in parallel. | Owner acknowledgement and status |
| 1.5 Verify | IS and KM | Confirm accounts, records, permissions, and collaboration spaces. Record unresolved exceptions. | Completion evidence or exception |

## Representative pathway logic

- Existing identity, abbreviated route: verify active accounts and training, then update local access and records.
- New or inactive identity, full route: complete enterprise prerequisites before local setup.
- Blocked request, exception route: record the blocker, assign an owner, and resume when evidence is available.

## Design principle

*Give each reader the shortest accurate path through the process while making ownership, dependencies, and completion evidence visible enough for every supporting team to coordinate.*

SAMPLE 03

# Automated Intake and Tracking Workflow

| | |
|---|---|
| **PROBLEM** | A manual deliverable-intake process made identifier control, metadata quality, attachment handling, status visibility, and user confirmation difficult to manage consistently. |
| **MY ROLE** | Designed the intake model and user experience, built the automation, tested the end-to-end workflow, wrote user and administrator messages, and documented failure handling. |
| **AUDIENCE** | Personnel submitting deliverables and administrators responsible for maintaining the official tracking register. |
| **TOOLS** | Microsoft Forms, Power Automate, SharePoint Lists, Microsoft Teams, and automated email notifications. |

## Guided workflow

| 1. SUBMIT | 2. VALIDATE | 3. RECORD | 4. PROCESS | 5. NOTIFY |
|---|---|---|---|---|
| Guided form | Required data | ID and metadata | Attachments | User and admin |

## Resulting self-service capability

- Gives users one guided submission interface instead of an informal handoff.
- Writes structured submission data to a centralized register embedded in the team workspace.
- Maintains sequential report identifiers and traceability across submissions.
- Processes attachments and associates them with the correct record.
- Confirms successful submission, communicates meaningful status changes, and gives administrators actionable recovery instructions when a transaction fails.

## Content design decisions

- Messages trigger only at meaningful state changes, reducing noise while preserving visibility.
- Subject lines and opening sentences identify the transaction and current state immediately.
- User-facing messages explain what happened and what, if anything, the reader must do next.
- Administrator alerts identify the failure point and provide concrete recovery steps.
- Instructions vary by audience and workflow state rather than forcing every reader through the same content.

## Hands-on documentation model

I built the system, tested its paths, diagnosed failures, and wrote the guidance around observed behavior. That loop lets the documentation account for the actual interface, dependencies, edge cases, and support burden instead of describing an idealized process.

SAMPLE 03 | ADAPTED EXCERPT

# Representative User Messages and System Guidance

The original workflow delivered short, action-oriented messages at each meaningful state change. These sanitized reconstructions preserve the original communication pattern.

| EVENT | AUDIENCE | REPRESENTATIVE MESSAGE |
|---|---|---|
| Submission accepted | Submitting user | Your deliverable was received and recorded as [Report ID]. The submission includes [file count] attachment(s). If corrections are required, the administrator will contact you. |
| Record created | Administrator | A new record was added to the tracking register. Review the assigned identifier, metadata, and attachment status. |
| Processing exception | Administrator | The system received submission [ID] but could not complete [processing step]. Follow the recovery steps below, then confirm the record is complete. |
| Missing information | Submitting user | Your submission could not be completed because a required field or attachment is missing. Return to the intake form, provide the listed item, and resubmit. |

## Additional documentation experience

- Authored a 60-plus-page information-architecture guide and converted high-use concepts into diagrams, quick-reference materials, and one-page graphics.
- Produced more than 15 instructional videos, including audience analysis, original scripts, instructional graphics, narration, audio editing, and final production.
- Reverse-engineered a customized ServiceNow environment, translated its departures from the expected operating model, configured thousands of work items for validation, and wrote adoption guidance.
- Documented automated workflows spanning Microsoft Forms, SharePoint Lists, Teams, Power Apps, and Power Automate, including validation rules, role-specific notifications, dependencies, and failure states.

## Public documentation project

What Is LangChain? - Live Documentation Site

## Contact

Declan Goodwin | Fredericksburg, Virginia | Declan.goodwin1252@gmail.com | linkedin.com/in/declan-goodwin